GROUP: FD2

DOMAIN 2

SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. Introduction:

This section "Upstart" is introducing the strategic solution of the identified problem statement highlighted in the Product Requirements Document.

1.1. Purpose:

*The System Analysis Document (SAD) outlines the technical scope, system architecture, and design decisions behind the **Upstart** Finance Layer. It serves as a reference for understanding how the system processes user financial data, integrates AI-based reasoning, and delivers structured financial insights through an interactive application.*

So, the key element of system that has been covered are as followed:

***1. Architecture**:*

The system is designed as a cross-platform MVP with a layered architecture consisting of:

- *Frontend layer supporting both web and mobile interfaces (React / React Native via Expo Go)*
- *Backend API layer handling data processing, storage, and tool-based AI requests*
- *AI layer powered by Z.AI GLM responsible for financial reasoning and insight generation*

These components are designed to be modular and loosely coupled, allowing independent development and future scalability.

2. Data Flows:

User financial data flows through the system in the following manner:

- *Users input financial data (income, expenses, debt, savings goals) using natural language or structured input*
- *The backend processes and stores this data in a persistent storage layer*
- *The GLM model retrieves structured financial context via system tools for analysis*
- *AI-generated insights are returned to the frontend and displayed in modules such as Dashboard, Debt Planner, and AI Advisor*

This ensures continuous synchronization between user input, stored data, and AI-driven financial insights.

3. Model Process:

The AI model operates through a grounded reasoning process:

- *It retrieves relevant financial data using system tools (income, expenses, debt summaries)*
- *It interprets user intent such as saving, debt reduction, or spending control*

© UMHackathon 2026

- *It generates contextual financial recommendations strictly based on available data*
- *Outputs are validated to reduce hallucination before being displayed to the user*

This ensures that all AI responses remain accurate, explainable, and grounded in real user financial records.

4. ***Role of Reference:***

This document serves as a technical reference for developers, testers, and system designers. It defines the high-level architecture, data flow structure, and AI interaction model to ensure consistent implementation and testing across all system components.

1.2. Background:

Many informal workers and low-income individuals manage finances using cash-based, unstructured methods without formal tracking systems. This leads to poor financial visibility, uncontrolled spending, and difficulty managing debt. The ***Upstart*** *addresses this problem by transforming unstructured financial behavior into structured insights using AI-driven financial analysis.*

1. ***Previous Version:***

   Traditional financial tracking systems rely heavily on manual entry through spreadsheets or rigid form-based inputs. These systems are often complex, time-consuming, and unsuitable for users with low financial literacy or irregular income patterns. They also lack intelligent financial reasoning capabilities and require users to manually interpret raw data.

2. ***Changes in Major Architectural Components:***

   The system introduces the following key improvements:

   - *Integration of Z.AI GLM for financial reasoning and advisory generation*
   - *Natural language input for financial data entry instead of rigid forms*
   - *Automated categorization of income, expenses, and debt*
   - *Real-time financial health scoring and insights generation*
   - *Cross-platform support (web + mobile via Expo Go) using a shared codebase*
   - *Modular dashboard replacing static spreadsheet-based financial tracking*

© UMHackathon 2026

1.3. Target Stakeholder

Stakeholders	***Roles***	***Expectations***
Users (Informal Workers / Vendors)	*Input financial data and use AI insights*	*Simple, fast financial tracking with clear and actionable advice*
AI System (GLM Layer)	*Processes financial data and generates insights*	*Accurate, grounded, and context-aware financial recommendations*
Development Team	*Builds and maintains the system*	*Clear API contracts, modular architecture, and maintainable codebase*
QA Team	*Validates system correctness and AI behavior*	*Defined test coverage, edge case handling, and reliable AI outputs*

© UMHackathon 2026

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	*Cross-platform Financial Intelligence Application (Web + Mobile via Expo Go)*
Architecture	*Client–Server Architecture with AI Service Layer (Z.AI GLM Integration)*

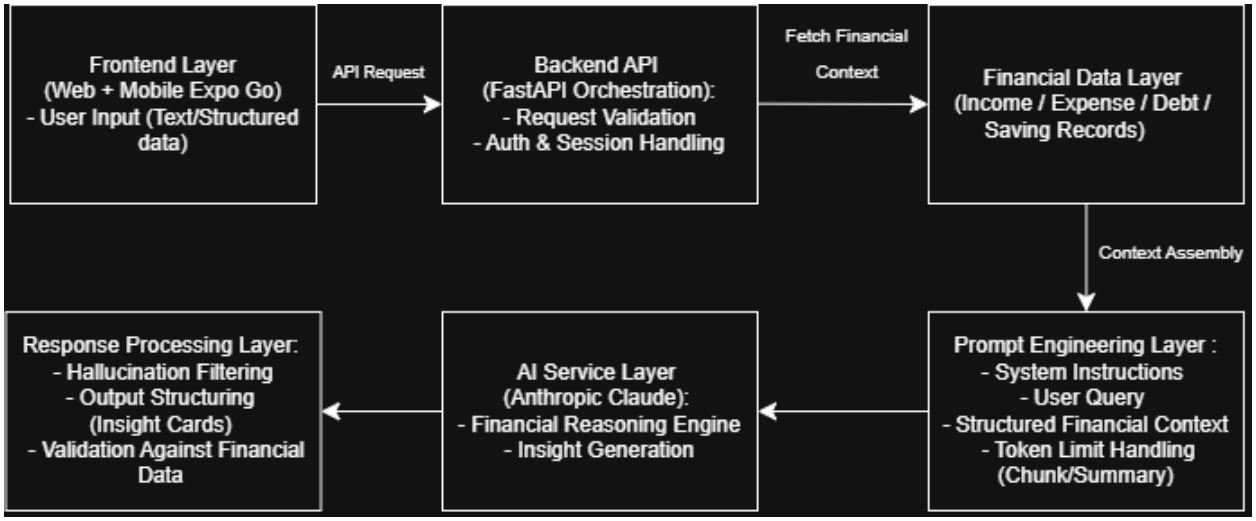
The system consists of two primary clients (web and mobile) connected to a backend API layer. The backend manages financial data processing, storage, and orchestration of AI requests. The AI layer is implemented as an external service using Anthropic Claude, which processes structured financial context and returns advisory insights.

2.1.2 LLM as Service Layer

The system integrates Anthropic Claude as an external large language model provider within a dedicated AI service layer. This layer functions as the core reasoning engine for financial advisory generation, operating strictly on structured financial context retrieved from backend services.

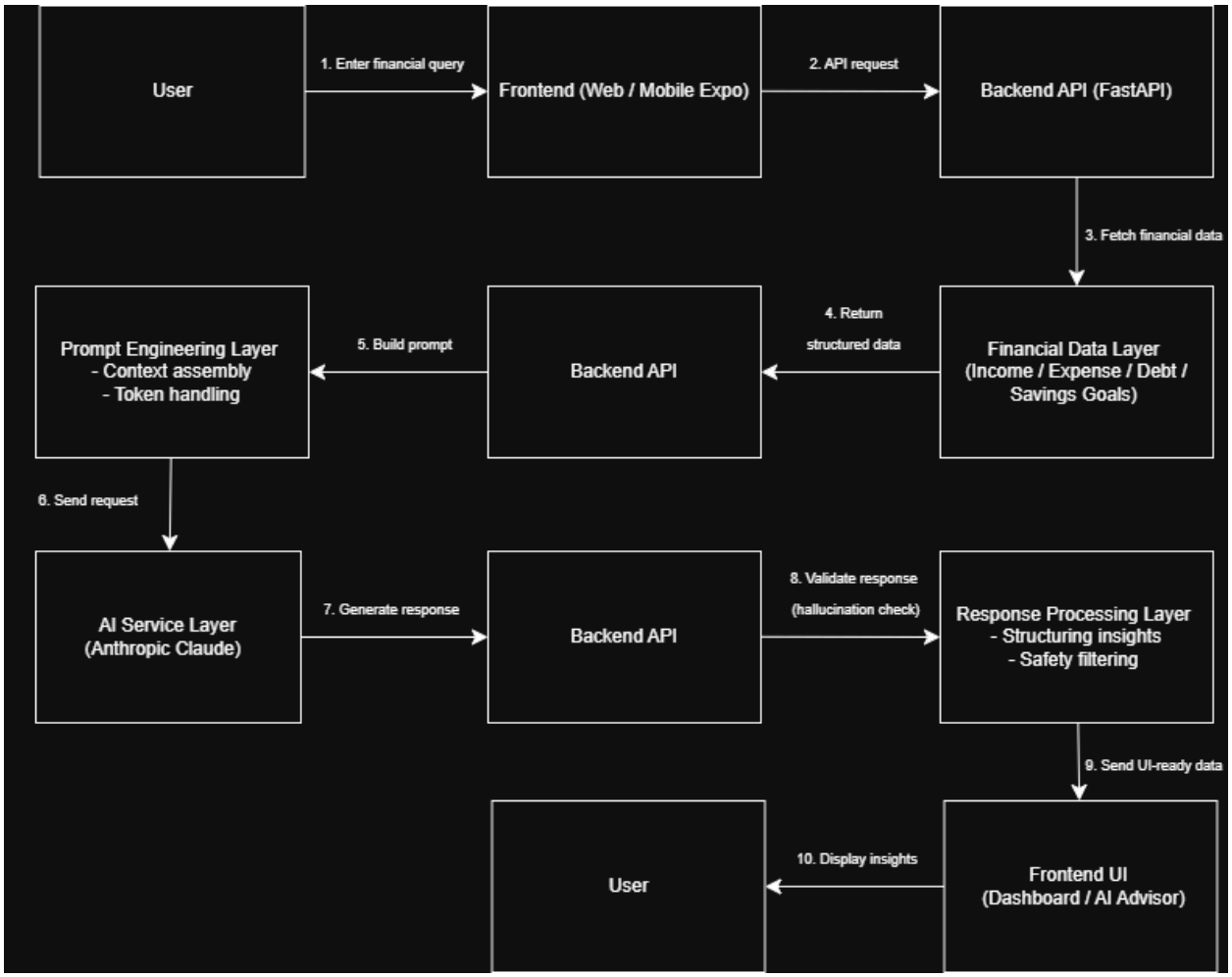
The design ensures separation between application logic and AI inference, allowing flexibility for future model replacement or multi-model expansion.

2.1.3 Dependency Diagram



© UMHackathon 2026

2.2.2. Sequence Diagram

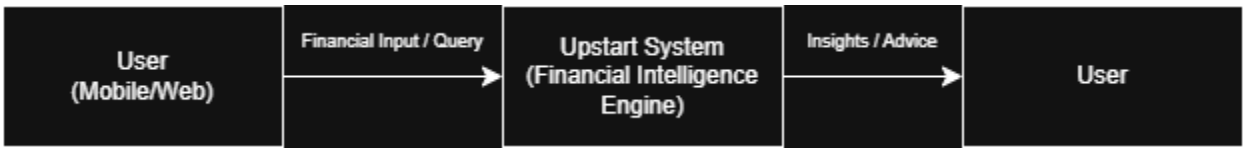


2.2. Technological Stack

- 2.2.1. *Frontend: React (Web) + React Native (Expo Go Mobile)*
- 2.2.2. *Backend: FastAPI (RESTful API services)*
- 2.2.3. *AI Service Layer: Anthropic Claude (API-based integration)*
- 2.2.4. *Database: Lightweight persistent storage (JSON / SQL-based depending on deployment)*
- 2.2.5. *Cloud/Deployment: Cloud-hosted backend with API architecture*

2.3. Key Data Flows

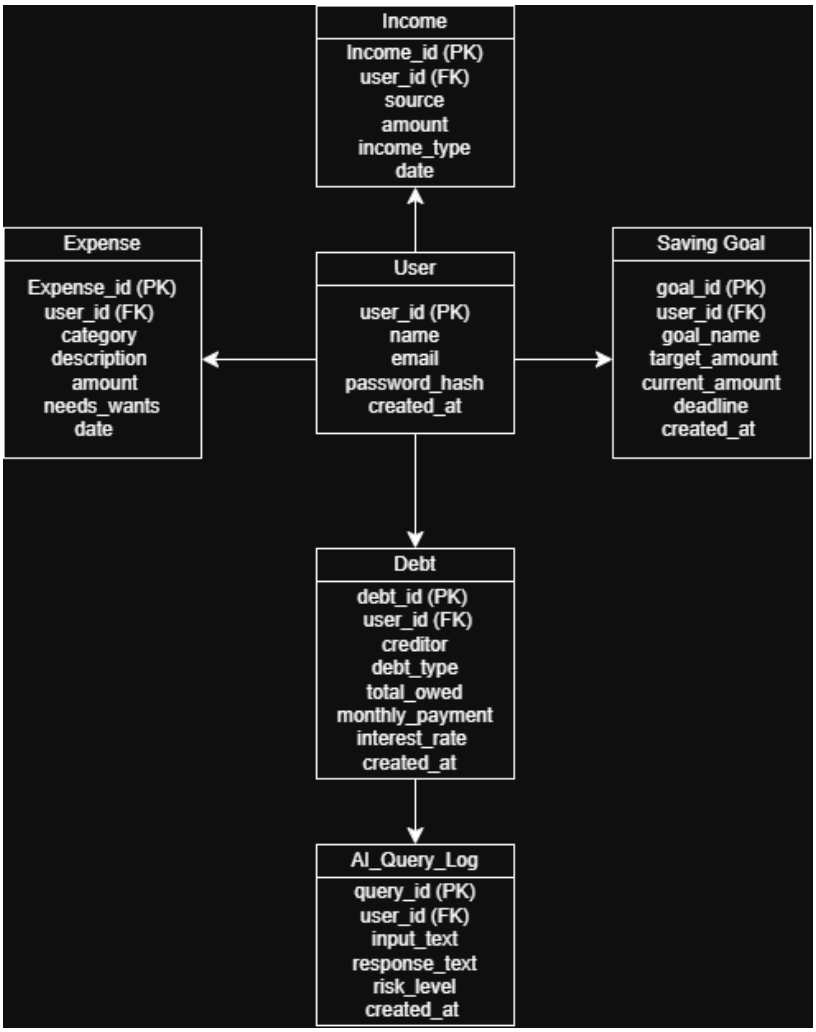
2.3.1.1. Data Flow Diagram (DFD)



© UMHackathon 2026

Email: umhackathon@um.edu.my

2.3.2. Normalized Database Schema



© UMHackathon 2026

Email: umhackathon@um.edu.my

3. Functional Requirements & Scope

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

3.1. Minimum Viable Product:

3.1.1. Table of MVP

#	Features	Description
1	Financial Data Logging (Income & Expense)	Users can log daily income and expenses using natural language or quick input. The system automatically categorises transactions into structured financial records.
2	Debt Management Dashboard	Users can input multiple debts and view a consolidated breakdown including total debt, monthly payments, and repayment priority suggestions.
3	AI Financial Advisor (GLM Integration)	The system uses GLM (Anthropic Claude via API layer) to analyse financial data and generate actionable insights such as spending alerts, savings opportunities, and debt recommendations.
4	Financial Health Score	The system calculates a real-time financial health score (0–100) based on income stability, debt ratio, savings buffer, and spending behaviour.
5	Savings Goals Tracking	Users can set simple financial goals (e.g., emergency fund, school fees) and track progress over time with AI-based surplus suggestions.

3.2. Non-Functional Requirements (NFRs)

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system. [e.g. table in below]

Quality (e.g.)	Requirements (e.g.)	Implementation (e.g)
Scalability	The system must support multiple users with concurrent financial logging and AI requests during peak usage periods.	Stateless backend design with scalable API layer and cloud-based deployment (e.g. autoscaling-ready architecture).

© UMHackathon 2026

Reliability	*Financial data must not be lost or duplicated. Every transaction must be stored consistently with high integrity.*	*Atomic database transactions and validation layer before commit operations.*
Maintainability	*The system must allow independent updates to AI layer, backend services, and frontend modules without breaking the system.*	*Modular architecture with separated services (Finance, AI, Analytics) and independent deployment pipeline.*
Token Latency	*AI responses must be returned within acceptable time for user experience.*	*GLM API calls are limited with timeout thresholds (≤5s) and asynchronous loading states in UI.*
Cost Efficiency	*Token usage must be controlled to avoid excessive LLM cost per user session.*	*Token usage must be controlled to avoid excessive LLM cost per user session.*

3.3. Out of Scope / Future Enhancements

The following features are planned for future development but are not included in the UMHackathon 2026 MVP:

- *Peer-to-peer financial sharing or family-linked budgeting accounts*
- *Integration with banking APIs for real-time transaction syncing*
- *Automated bill payments or financial execution actions*
- *Investment advisory features (stocks, crypto, or portfolio management)*
- *Multi-language voice assistant for full conversational financial management*
- *Exportable financial reports for formal loan applications or institutions*

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. ***Grayscale Rollout & A/B Testing****: New features (e.g., AI financial advisor updates or improved expense categorisation) will be deployed gradually using a grayscale rollout strategy. Initially, features will be released to a small subset of users (e.g., 5–10%) to monitor system stability, AI response quality, and user interaction behaviour. If no critical issues are detected, the rollout will progressively expand to 100% of users. After full deployment, A/B testing will be conducted to compare different AI prompt strategies or UI variations to determine which produces better financial decision-making outcomes and user engagement.*

© UMHackathon 2026

4.1.2. ***Emergency Rollback (ER) & Golden Release**: In the event of system instability such as incorrect AI financial recommendations, API failures, or significant performance degradation, an Emergency Rollback mechanism will be triggered. The system will automatically revert to a stable "Golden Release" version of the application. This ensures continuity of service while preventing users from receiving unreliable financial advice. Rollbacks are supported by versioned deployments and CI/CD pipeline snapshot backups.*

4.1.3. Priority Matrix:

4.1.3.1.

Priority	*Condition*	*Action*
P1 (Critical)	*AI produces incorrect financial advice affecting debt/savings recommendations OR system downtime > 5 minutes*	*Immediate rollback to Golden Release + disable AI module temporarily*
P2 (High)	*API latency exceeds 5 seconds consistently OR error rate > 2%*	*Trigger performance alert + activate fallback cached responses*
P3 (Medium)	*UI glitches or incorrect categorisation of transactions*	*Log issue + schedule patch in next deployment cycle*
P4 (Low)	*Minor UI inconsistencies or non-critical suggestion mismatches*	*Fix in backlog without system interruption*

4.2. Monitoring:

4.2.1. *The system will be monitored using real-time logging and health metrics tracking. Key indicators include API latency, AI response accuracy rate, error frequency, and database consistency checks. Alerts will be triggered when thresholds are exceeded (e.g., latency > 5s or error rate > 2%). In such cases, fallback mechanisms such as cached responses or simplified rule-based outputs will be activated to maintain system stability. Continuous monitoring ensures both financial data integrity and AI response reliability.*

4.3. Assumptions:

- *Users have stable internet connectivity during financial data entry and AI interactions*
- *Users input data truthfully and consistently for accurate financial analysis*
- *Device time and regional settings are correctly configured for transaction logging*
- *External APIs (e.g., GLM API) remain available with standard rate limits*
- *Users understand basic financial categories such as income, expense, and debt*
- *No direct banking integration is required in the MVP phase*

© UMHackathon 2026

4.4. External Dependencies:

Do list all the external tools that are applied to your system to function.

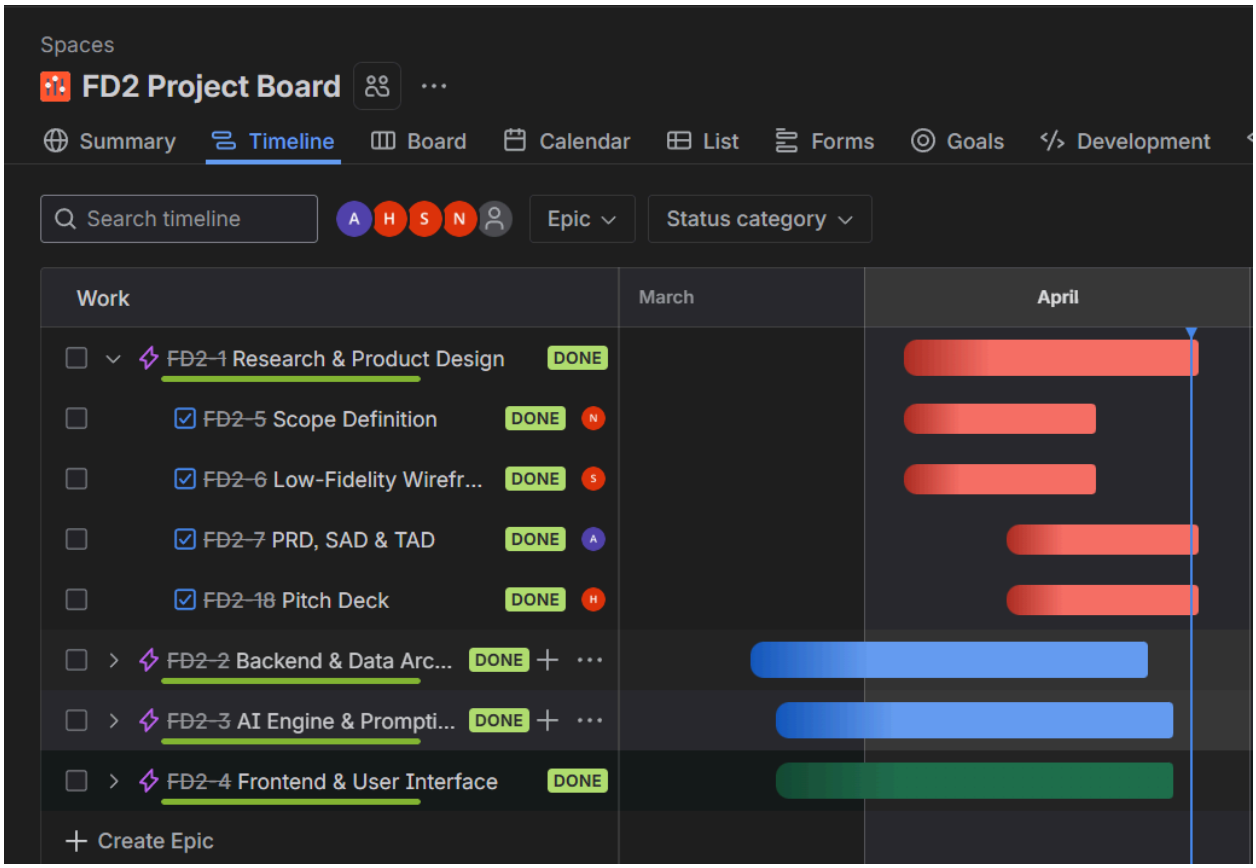
Tools	*Purpose*	*Risks*
Z.AI GLM (Anthropic Claude via API)	*Core AI engine for financial reasoning, categorisation, and recommendation generation*	*High – API downtime or rate limits may degrade AI advisory features; mitigated using caching and fallback rule-based logic*
Firebase / Backend Database (or equivalent)	*Stores user financial records including income, expenses, debts, and goals*	*Medium – potential data sync issues; mitigated using backup and atomic transactions*
Expo (React Native Framework)	*Mobile deployment for cross-platform application (Android + iOS)*	*Low–Medium – build or compatibility issues across devices*
Cloud Hosting (e.g. AWS / Vercel)	*Backend deployment and API hosting*	*Medium – downtime risk mitigated via redundancy and auto-scaling*

© UMHackathon 2026

5. Project Management & Team Contributions

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. Project Timeline:



5.2. Team Members Role:

Team Member	*Role*	*Responsibilities*
Naim	*AI Integration Engineer / Backend Lead*	*- Designs and implements FastAPI services, database schema, and financial data APIs (income, expense, debt, goals).* *-Handles GLM (Anthropic Claude via API) integration, prompt engineering, context building, and AI response validation logic.*
Akmal	*System Architect & Data Designer*	*Designs system architecture, DFD, ERD, data flow, and*

© UMHackathon 2026

		ensures normalization and consistency across system components.
Shahril	*Frontend / Mobile Developer*	*Develops Expo (React Native) mobile interface including dashboards, logging screens, and AI advisor UI.*
Haneef	*QA & DevOps Support*	*Handles testing strategy (unit, integration, AI testing), CI/CD setup, and monitoring/rollback configuration.*

5.3. Recommendations:

Performance Optimization

- *Implement response caching (Redis or in-memory cache) for frequently accessed financial summaries and repeated AI queries*
- *Use prompt compression and chunking strategies to reduce token usage in GLM calls*
- *Apply asynchronous processing for AI calls to prevent UI blocking*

Scalability Improvements

- *Design backend as stateless API services to allow horizontal scaling*
- *Separate AI service layer from core financial processing logic for independent scaling*
- *Use cloud-based deployment (auto-scaling capable) for handling peak user traffic*

Reliability Enhancements

- *Implement retry mechanisms with exponential backoff for GLM API failures*
- *Use fallback rule-based financial insights when AI service is unavailable*
- *Maintain database backup and transaction logging for financial data integrity*
- *Introduce health monitoring and alerting system for API latency and error rates*

© UMHackathon 2026

© UMHackathon 2026

Email: umhackathon@um.edu.my